

---

## Day 36 – Prototype Design Pattern

---

### Introduction

---

The **Prototype Design Pattern** is a **Creational Design Pattern**.

Its main purpose is to **create new objects by copying (cloning) an existing object instead of creating a new object from scratch**. This is useful when object creation is expensive or time-consuming.

---

### Why Prototype Design Pattern?

---

Normally, whenever we need a new object, we do:

```
A* obj1 = new A();  
A* obj2 = new A();  
A* obj3 = new A();
```

Every time,

- Memory is allocated.
- Constructor executes.
- Initialization happens again.

If we need hundreds or thousands of similar objects,

this becomes expensive.

---

### Problem Statement

---

Suppose we already created one object.

```
Class A  
|  
▼  
Object A1
```

Now we need another object having almost the same properties.

Normally,

```
A* a1 = new A();  
  
A* a2 = new A();
```

Again,

- Memory allocation
- Constructor call
- Initialization

Everything repeats.

## Why is Object Creation Expensive?

Object creation may involve expensive operations like:

### 1. Database Connection

```
Application  
↓  
Connect Database  
↓  
Create Object
```

### 2. Complex Calculations

Example

```
Large Mathematical Computation  
↓  
Object Initialization
```

### 3. Loading Large Files

Example

Configuration File



Images



Models



Object

Creating the same object repeatedly wastes both time and resources.



## Solution – Prototype Pattern

Instead of creating a new object,

create **one object** first.

Whenever another similar object is needed,

**clone the existing object.**

Original Object



clone()



Copied Object

Only the required properties are modified afterward.



## Real Example – NPC (Non-Player Character)

We explain the Prototype Pattern using an **NPC (Non-Player Character)** example. NPCs are background characters in games that perform predefined tasks (for example, pedestrians in GTA).

Examples:

- GTA pedestrians
- Shopkeepers
- Soldiers
- Villagers
- Police

Suppose a game needs

1000 NPCs

All NPCs are almost identical.

Example

Health = 100

Power = 50

Weapon = Gun

Dress = Same

Only

Name

Position

Color

Weapon

may change.

**Without Prototype**

Create NPC1

Create NPC2

Create NPC3

...

```
Create NPC1000
```

Every object performs expensive initialization.

### With Prototype

```
Create NPC1
```

↓

```
Clone NPC1
```

↓

```
Clone NPC2
```

↓

```
Clone NPC3
```

↓

```
Update Small Changes
```

Only one expensive object is created.

The remaining objects are copied.

This greatly reduces object creation time.



## How Prototype Solves the Problem

### Step 1

Create the first object.

```
NPC* npc1 = new NPC();
```

### Step 2

Need another NPC?

Instead of

```
NPC* npc2 = new NPC();
```

Do

```
NPC* npc2 = npc1->clone();
```

### Step 3

Update only the required properties.

Example

```
npc1  
  
Health =100  
  
Power =50  
  
Name ="Soldier"  
  
↓  
  
Clone  
  
↓  
  
npc2  
  
Health =100  
  
Power =50  
  
Name ="Police"
```

Everything else remains the same.



## Cloning Concept

---

Prototype Pattern is based on the idea of **copying an existing object**.

Instead of constructing a new object,

we duplicate an existing one.

Original Object

↓

clone()

↓

New Object

The cloned object contains the same data as the original object.

---

## How Does clone() Work?

---

We mention that cloning is implemented using a **Copy Constructor**.

Instead of manually copying each property,

the copy constructor creates a new object using the existing object.

Example

```
NPC(const NPC& other)
{
    name = other.name;
    health = other.health;
    power = other.power;
}
```

Now

```
clone()
```

simply returns

```
return new NPC(*this);
```

---

## Three Constructors

---

The notes mention three common constructors:

## 1. Default Constructor

```
NPC()
```

No parameters.

## 2. Parameterized Constructor

```
NPC(string name,int health,int power)
```

Creates an object with specific values.

## 3. Copy Constructor

```
NPC(const NPC& other)
```

Creates a new object by copying another object.

Prototype Pattern mainly uses the **Copy Constructor**.

---



## Key Idea

---

Instead of

Create

Create

Create

Create

Use

Create Once

↓

Clone Many Times

This saves

- Time
- Memory allocation cost
- Expensive initialization



## Copy Constructor

---

The **Prototype Design Pattern** mainly works using the **Copy Constructor**.

Instead of creating a completely new object,

we create a copy of an existing object.

### What is a Copy Constructor?

A Copy Constructor creates a **new object by copying another object of the same class**.

Syntax

```
ClassName(const ClassName& other)
{
    // Copy data
}
```

### Example

Suppose we have an NPC object.

```
NPC npc1("Soldier", 100, 50);
```

Now instead of creating another NPC manually,

```
NPC npc2(npc1);
```

The copy constructor copies all the values from **npc1** into **npc2**.

### Working

```
NPC1
Name = Soldier
```

```
Health =100

Power =50

      |
      | Copy Constructor
      ▼

NPC2

Name = Soldier

Health =100

Power =50
```

Both objects are different,  
but their data is the same.



## Clone Method

Instead of exposing the Copy Constructor directly,  
Prototype Pattern provides a **clone()** method.

Example

```
NPC* clone()
{
    return new NPC(*this);
}
```

Here,

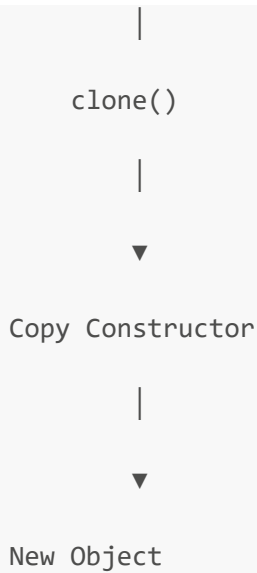
```
*this
```

means

Copy the **current object**.

## Working Flow

Existing Object

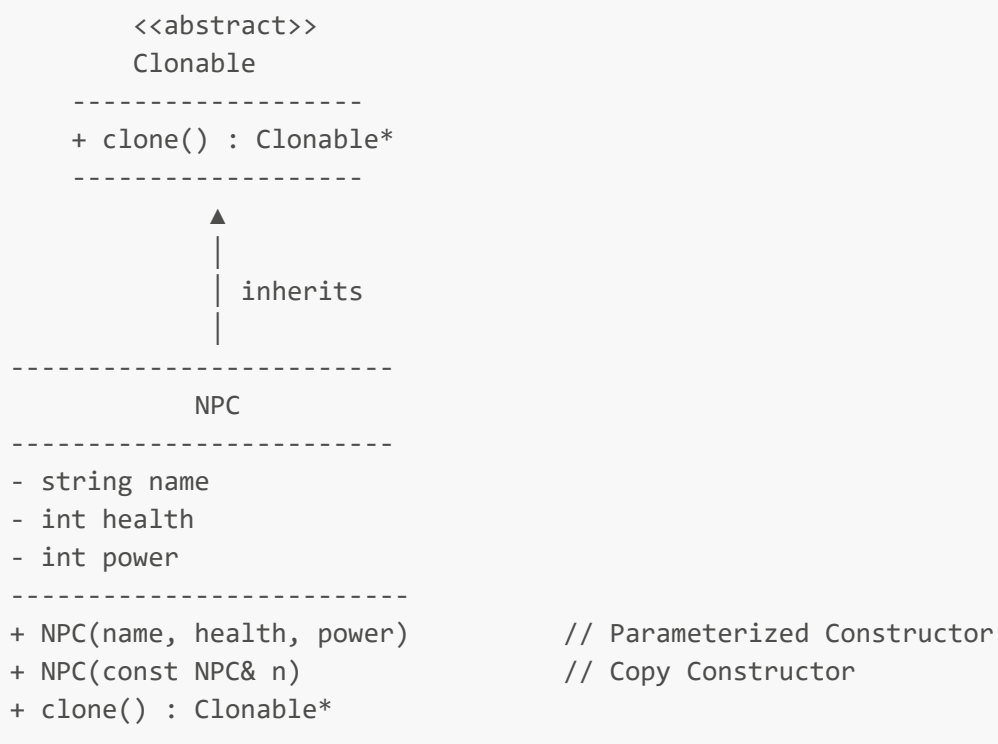


The client never calls the copy constructor directly.

It simply calls

```
clone()
```

## NPC UML Diagram



# ■ Prototype UML

## 1. Clonable (Abstract Class)

```
<<abstract>>
Clonable
-----
clone()
```

- **Clonable** is the **Prototype interface**.
- Any class that inherits it **must implement clone()**.
- It makes the class **cloneable**.

## 2. NPC Class

```
NPC
-----
name
health
power
-----
NPC(name,h,p)
NPC(const NPC& n)
clone()
```

NPC stores the object data and implements the **clone()** method.

## 3. Parameterized Constructor

```
NPC("Soldier",100,50);
```

- Creates the **first/original object**.

## 4. Copy Constructor

```
NPC(const NPC& n)
```

- Creates a **new object by copying** another NPC object.

Example:

```
npc1  →  npc2
```

```
Name      = Soldier
Health    = 100
Power     = 50
```

## 5. clone() Method

```
clone()
{
    return new NPC(*this);
}
```

- `*this` means **current object**.
- It calls the **Copy Constructor**.
- Returns a **new copied object**.

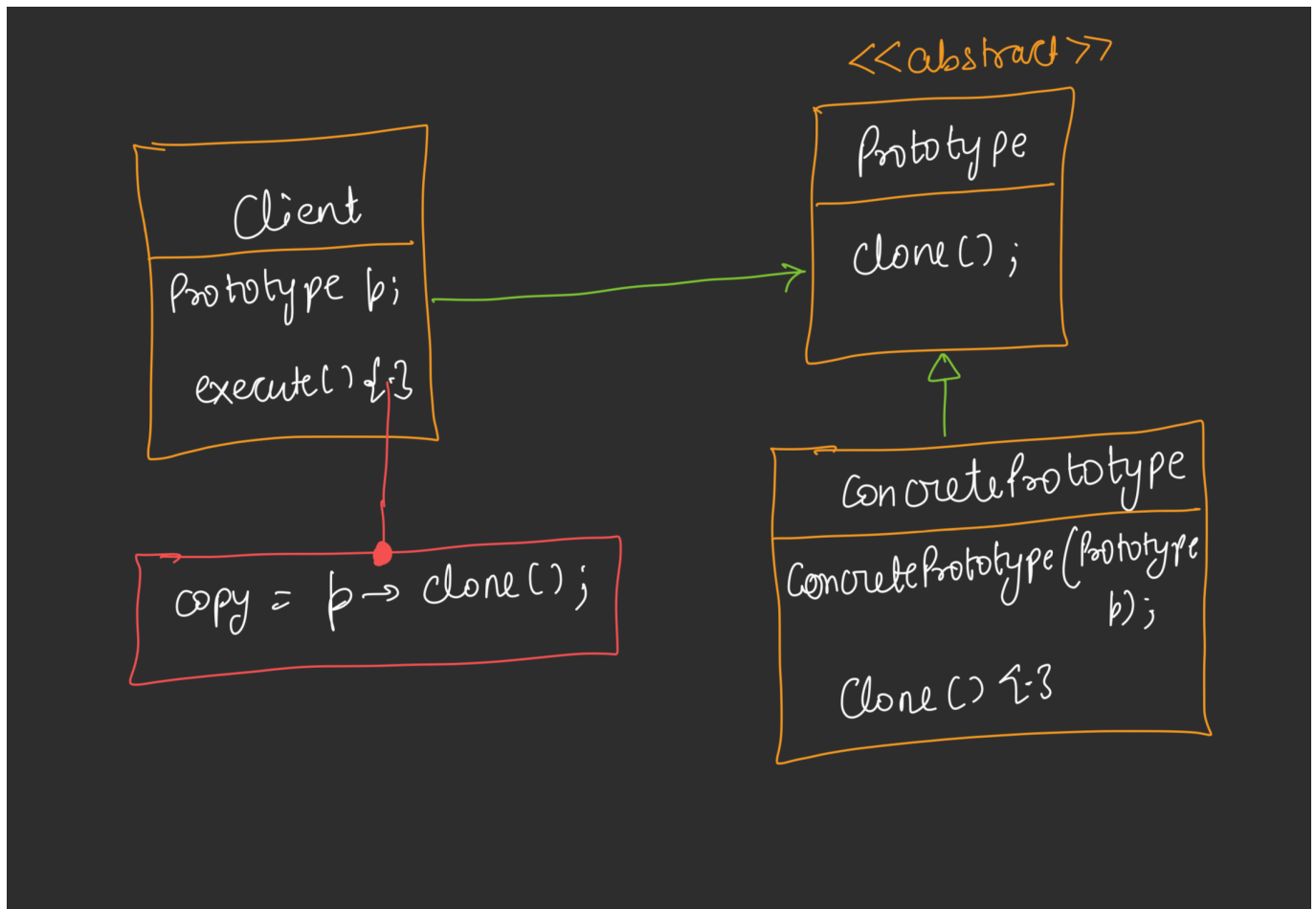
### Flow

```
Create Original NPC
    |
    ▼
npc1
    |
    clone()
    |
    ▼
Copy Constructor
    |
    ▼
New NPC (Copied Object)
```

---

### Standard UML

---



## Standard Definition

**Prototype Pattern** specifies the kinds of objects to create using a prototypical instance and creates new objects by copying this prototype.

### Flow

Client

|

clone()

|

Prototype

|

Returns

▼

New Object



## Prototype Flow

---

Client

|

Need New Object

|



Existing Prototype

|

clone()

|



Copy Constructor

|



New Object



## Why Not Create New Objects?

---

Without Prototype

Create Object

↓

Constructor

↓

Initialization

↓

Ready

Every object repeats the same expensive work.

With Prototype

Existing Object

↓

clone()

↓

Ready

Only one object is initialized.

The rest are copied.



## Real Life Examples

---

Prototype Pattern is useful when many similar objects are required.

Examples:

- Game NPCs
- Enemy Characters
- Bullets
- Trees
- Cars
- Documents/Templates
- Resume Templates
- PowerPoint Templates

Instead of creating everything from scratch,

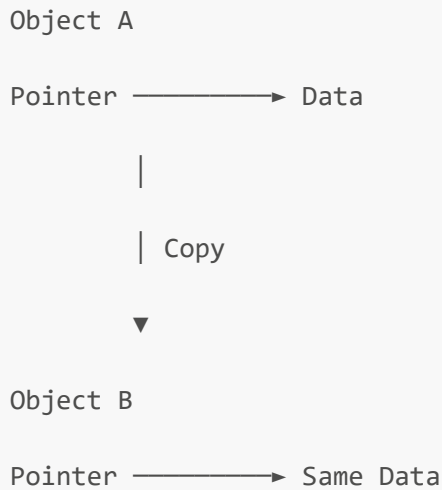
copy an existing object and modify only the required fields.



## Shallow Copy

---

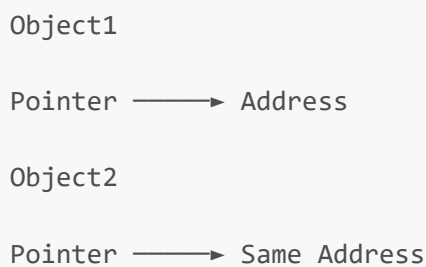
Shallow Copy copies only the **references (addresses)** of objects.



Both objects share the same memory.

If one object changes the shared data,  
the other object is also affected.

## Problem with Shallow Copy

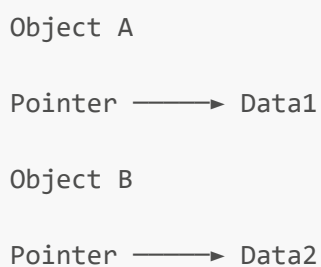


Deleting one object may cause problems because both point to the same memory.



## Deep Copy

Deep Copy creates **completely new memory**.



Each object owns its own copy.

Changing one object does not affect the other.

---

## Difference

| Shallow Copy              | Deep Copy               |
|---------------------------|-------------------------|
| Copies reference          | Copies actual data      |
| Shared memory             | Separate memory         |
| Faster                    | Slightly slower         |
| Unsafe for dynamic memory | Safe for dynamic memory |

---



## Advantages

---

- ✓ Faster object creation
  - ✓ Reduces expensive initialization
  - ✓ Less object creation cost
  - ✓ Easy to clone complex objects
  - ✓ Good for games
  - ✓ Reduces duplicate code
  - ✓ Improves performance
- 



## Disadvantages

---

- ✗ Cloning complex objects can be difficult
  - ✗ Deep Copy implementation may become complex
  - ✗ Circular references are difficult to clone
  - ✗ Every class must implement `clone()`
- 



## Interview Questions

---

Q1. What problem does Prototype solve?

It avoids creating expensive objects repeatedly by cloning an existing object.

Q2. Which constructor is mainly used?

## Copy Constructor

Q3. Difference between Copy Constructor and clone()?

- **Copy Constructor** copies an object internally.
- **clone()** is a public method that usually uses the copy constructor to create and return a new object.

Q4. When should Prototype be used?

When object creation is expensive and many similar objects are required.

Q5. What is the difference between Shallow Copy and Deep Copy?

- **Shallow Copy** copies references, so objects share the same memory.
- **Deep Copy** creates a separate copy of the data, so each object has independent memory.